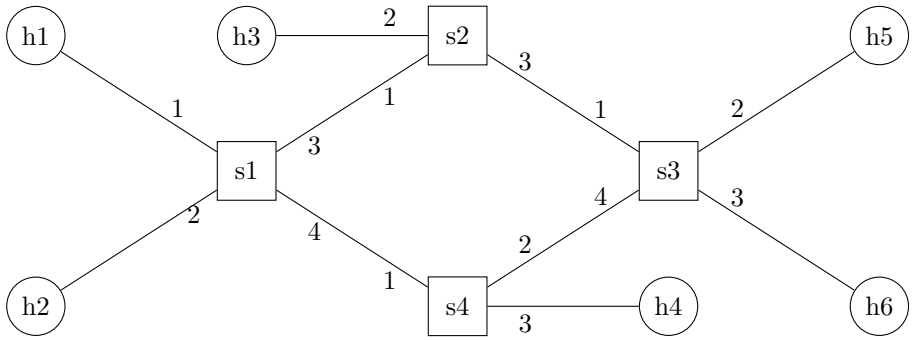
Software Defined Networking – 5 luglio 2019

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Nelle risposte usare per ogni pacchetto il seguente formato:

Nodo → interfaccia: MAC sorgente → MAC destinazione; ethertype; IP sorgente → IP destinazione; TTL; protocollo; porta sorgente → porta destinazione;

Negli switch sono preinstallate le regole in Figura 1.

L'host h1 porta 1234 apre una connessione TCP verso h6 porta 80.

(a) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

Soluzione: s1 → 2,3,4: MAC1 → MAC6; IPv4; IP1 → IP6; 200; TCP; 1234 → 80;

(b) Dire quanti pacchetti arrivano in s4 e che valore ha il campo TTL.

Soluzione: Uno con TTL=200.

L'host h2 porta 3322 invia un datagramma UDP verso h5 porta 122.

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione: Arrivano due pacchetti (uno da s1 e uno da s3), ciascuno delle quali causa una packet_in.
`packet_in`(MAC2 → MAC5; IPv4; IP2 → IP5; 200; UDP; 3322 → 122)
Il controllore risponde installando una regola, che però non viene usata per il pacchetto che ha causato la packet in.

(d) Dire quanti pacchetti arrivano in h5 e con che TTL.

Soluzione: Uno con TTL=200.

(e) L'host h5 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione: Non arrivano pacchetti in s2.

(f) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

Soluzione: Non arrivano pacchetti in s1.

L'host h2 porta 3322 invia un datagramma UDP verso h6 porta 123.

(g) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione: Arrivano due pacchetti (uno da s1 e uno da s3), ciascuno delle quali causa una packet_in.
`packet_in`(MAC2 → MAC6; IPv4; IP2 → IP6; 200; UDP; 3322 → 123)
Il controllore non risponde.

L'host h3 porta 7777 apre una connessione TCP verso l'host h4 porta 245.

(h) Scrivere una regola da inserire in s4 che permetta ai pacchetti della connessione di raggiungere la destinazione. La regola non deve essere usata per alcuno dei pacchetti di cui alle precedenti domande.

Soluzione: Varie soluzioni possibili. Per esempio:

Switch s4

Priority	Match	Action
5	eth_dst = MAC3	Output(3)

(i) Scrivere una regola da inserire in s4 che permetta ai pacchetti di risposta di raggiungere la destinazione. La regola non deve essere usata per alcuno dei pacchetti di cui alle precedenti domande.

Soluzione: Varie soluzioni possibili. Per esempio:

Switch s4		
Priority	Match	Action
5	eth_dst = MAC4	Output(1)

Figura 1: Regole iniziali

Switch s1

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_dst = MAC6	Output(FLOOD)
5	eth_dst = MAC5	Output(FLOOD)

Switch s2

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_dst = MAC6 eth_type = IPv4 ip_proto = TCP	DecTTL Output(FLOOD)
5	eth_dst = MAC5 eth_type = IPv4 ip_proto = TCP	DecTTL Output(FLOOD)

Switch s3

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_dst = MAC6 eth_type = IPv4 ip_proto = UDP	Output(FLOOD)
5	eth_dst = MAC5 eth_type = IPv4 ip_proto = UDP	Output(FLOOD)

Switch s4

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_dst = MAC6	Output(FLOOD)
5	eth_dst = MAC5	Output(FLOOD)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (datapath.id == 2 and pkt_udp is not None and pkt_ipv4.dst == IP5):
        match = parser.OFPMatch(eth_type=ETH_TYPE_IP, ipv4_dst = pkt_ipv4.src,
                ip_proto=IPPROTO_UDP, udp_dst=pkt_udp.src_port)
        actions = [ parser.OFPSetField(udp_dst = pkt_udp.src_port+3),
                parser.OFPActionOutput(1)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=10,
            match=match, instructions=inst)
        datapath.send_msg(mod)
        return
```

Esercizio 2

Un filtro di Bloom *a conteggio* è costituito da un array $A[_]$ di m numeri interi e k funzioni hash. Nel filtro *a conteggio* ogni volta che viene inserito un elemento x, le celle di A in posizione $h_i(x)$ con $1 \leq i \leq k$ sono incrementate di 1. Nel filtro *a conteggio* è possibile rimuovere un elemento decrementando le celle corrispondenti.

Un sistema di blocco di URL fraudolente usa un filtro di Bloom costituito da un array di $m = 2^{10}$ celle e $k = 4$ funzioni hash.

(a) Calcolare la probabilità di falso positivo dopo aver effettuato l'inserimento di 100 URL.

Soluzione:

$$\left(1 - e^{\left(-\frac{25}{64}\right)}\right)^4 = 1\%$$

(b) Calcolare la grandezza minima dell'array (in bit) per non avere overflow dopo 10 000 inserimenti.

Soluzione: Nel caso pessimo è incrementata sempre la stessa cella, che varrà $10000k$. Servono quindi $\lceil \log_2 40000 \rceil = 16$ bit per cella. In totale sono 16348 bit.

(c) Calcolare il numero minimo di celle dell'array per avere un probabilità di falso positivo di 10^{-4} dopo 10 000 inserimenti.

Soluzione:

$$\left(1 - \exp(-4 \times 10^4/m)\right)^4 < 10^{-4}$$

da cui $m > \frac{4 \times 10^4}{\log 0.9}$, ovvero $m > 379649$

Si considerino $m = 17$, $k = 2$ e la famiglia di funzioni hash $h_i(x) = 3i + 5x \bmod m \quad$ per $1 \leq i \leq k$

(d) Mostrare lo stato del filtro dopo l'inserimento di ciascuno dei seguenti elementi: 3, 7.

Soluzione:

$$\{1 \to 1, 4 \to 1\}$$

$$\{1 \to 1, 4 \to 2, 7 \to 1\}$$

(e) Verificare l'appartenza al dataset dell'elemento 41. Se è presente, rimuoverlo e mostrare lo stato del filtro.

Soluzione:

$$\{1 \to 1, 4 \to 1\}$$

Esercizio 3

Una matrice di commutazione crossbar di uno switch 4×4 usa un algoritmo per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza L. Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto. L'algoritmo di assegnazione dei ticket obbedisce alla seguente regola: *le interfacce di uscita scelgono sempre le interfacce di ingresso con l'identificativo minore.*

Ingresso 1: 4, 2, 1

Ingresso 2: 2, 1, 3

Ingresso 3: –, 3, 2

Ingresso 4: –, 4, 1

(a) Svolgere l'algoritmo Take-a-Ticket (TaT) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati.

Soluzione:

Round		Ingresso 1	2	3	4
1	Assegnazione	(1,1)	(3,1)	(2,1)	(1,2)
	Trasmissione	1	3	2	–
2	Assegnazione	(2,2)	(1,3)	(3,2)	–
	Trasmissione	2	–	3	1
3	Assegnazione	(4,1)	–	–	(4,2)
	Trasmissione	4	1	–	–
4	Assegnazione	–	(2,3)	–	–
	Trasmissione	–	2	–	4

(b) Si consideri il caso in cui i pacchetti diretti all'uscita 3 abbiano tutti lunghezza $2L$. Ripetere la simulazione dell'algoritmo TaT.

Soluzione:

Round		Ingresso 1	2	3	4
1	Assegnazione	(1,1)	(3,1)	(2,1)	(1,2)
	Trasmissione	1	3	2	–
2	Assegnazione	(2,2)	–	(3,2)	–
	Trasmissione	2	(3)	–	1
3	Assegnazione	(4,1)	–	–	(4,2)
	Trasmissione	4	1	3	–
4	Assegnazione	–	–	–	–
	Trasmissione	–	2	(3)	4

Facendo riferimento all'algoritmo TaT con assegnazione casuale, si consideri uno scenario di traffico uniforme e code sempre sature.

(c) Qual è la probabilità che sia attiva soltanto l'interfaccia di uscita numero 4?

Soluzione: Questo evento si verifica solo quando tutti gli ingressi chiedono di trasmettere verso l'uscita 4.

$$p = \frac{1}{4^4}$$

(d) Qual è la probabilità che siano attive almeno due interfacce di uscita? E' l'evento complementare all'evento in cui tutti gli ingressi chiedono di trasmettere verso la stessa uscita.

Soluzione:

$$p = 1 - \frac{4}{4^4}$$

Esercizio 4

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
64.0.0.0/2	*	80	*	TCP
64.0.0.0/3	128.0.0.0/2	80	22	TCP
*	96.0.0.0/3	*	53	UDP
64.0.0.0/2	*	*	123	UDP
128.0.0.0/2	64.0.0.0/2	23	*	UDP
96.0.0.0/3	64.0.0.0/3	25	80	*

(a) Definire il bitvector set per ciascun campo.

Soluzione:

Source Address	
64.0.0.0/2	101100
64.0.0.0/3	111100
96.0.0.0/3	101101
128.0.0.0/2	001010
*	001000

Destination Address	
64.0.0.0/2	100110
64.0.0.0/3	100111
96.0.0.0/3	101110
128.0.0.0/2	110100
*	100100

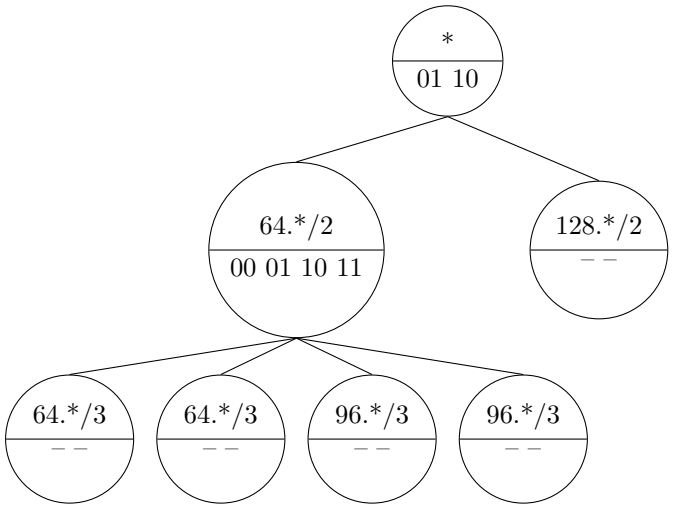
Source Port	
23	001110
25	001101
80	111100
*	001100

Dest. Port	
22	110010
53	101010
123	100110
80	100011
*	100010

protocol	
TCP	110001
UDP	001111
*	000001

(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie a passo 2. Disegnare il trie.

Soluzione:



(c) Si considerino i pacchetti (64.10.0.1, 96.0.0.2, 80, 53, UDP) e (97.50.0.1, 96.0.0.2, 80, 123, TCP). Applicare ad entrambi la procedura di BVLS per identificare gli eventuali match.

Soluzione: 1) 111100 AND 101110 AND 111100 AND 101010 AND 001111 = 001000
La terza regola.
2) 101101 AND 101110 AND 111100 AND 100110 AND 110001 = 100000
Il pacchetto soddisfa la regola 1.

(d) Si dica qual è la profondità massima dello unibit trie nei due casi IPv4 e IPv6.

Soluzione: Nel caso IPv4 32 bit. Nel caso IP64 128 bit.